

California Housing Project

Jan Wawrzynek

February 2026

1 Introduction

The goal of this project is create an ML model to predict the price of a house in California to an accuracy greater than that of Financial Analysts $\sim 30\%$ based on a number of features. The project is packaged using **Docker** for easy reproducibility. A server was set up using **FastAPI** to allow users to make individual predictions using the final ML model.

2 Data Exploration

From our feature correlation scatter plot. We can see that the median income feature is important for predicting the house price. However, from the plot we can see flat bands of median incomes, where it appears that the incomes were capped. I will work to remove these districts so as to prevent our model from learning these artificial patterns.

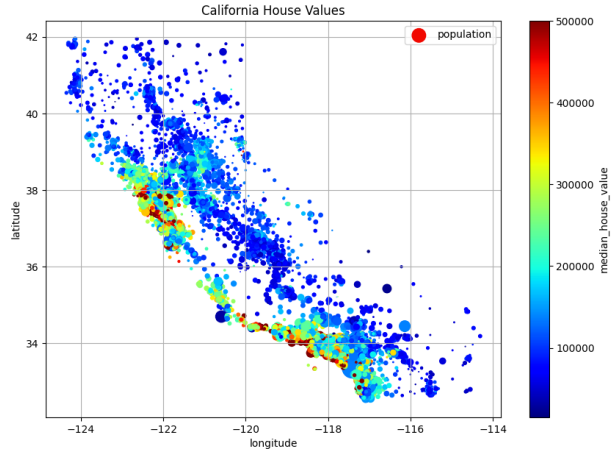


Figure 1: Map of data, with the size of data point scaling with population and median house price scaling with colour.

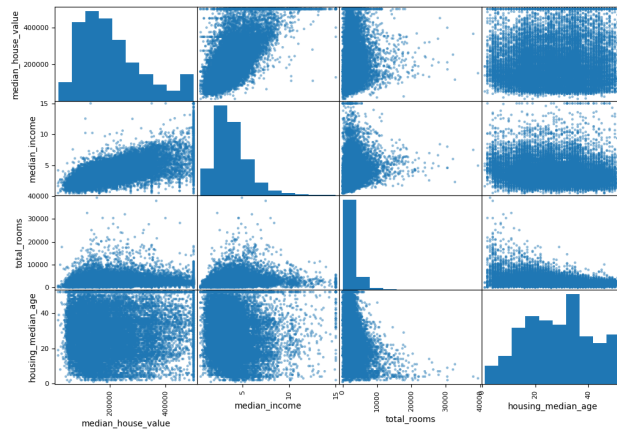


Figure 2: Scatter plots of interesting attributes to determine features correlated with our target.

2.1 Data Splitting

A key requirement of this project is to avoid "data snooping bias", which occurs when the model inadvertently has access to the test data and thus overfits the models parameters, resulting in poorer performance than expected. The split was performed using SKLearn's *train_test_split* function. 80% of the dataset

was assigned to the training data set and the remaining 20% was assigned to the testing data set, which we will not touch until we have finished tuning our model. The *train_test_split* function's hyperparameter was set to *stratify = income_cat*. To ensure that the data was split in a manner proportionally representative of the median incomes in the dataset.

We have information that the median income is an important feature for predicting house prices in California. Hence we want to ensure that the test set is representative of the various categories of incomes in the whole dataset. As the median income is a continuous numerical attribute, first I created an income category attribute. While most median income values are clustered around 1.5 to 6 (i.e., \$15,000-60,000), but some of the median incomes go far beyond that. It is crucial to have a sufficient number of instances of each stratum in the test dataset, as otherwise the estimate of each stratum's importance can be biased.

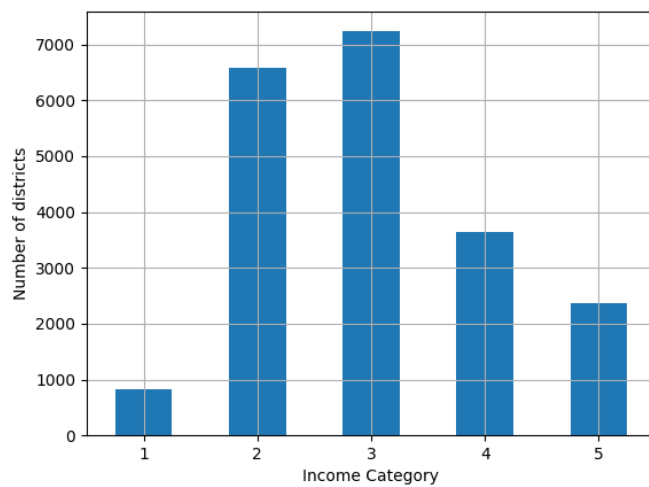


Figure 3: The continuous median income feature , categorised into bucketed income histogram.

3 Data Cleaning



Figure 4: Full vs Capped Value Filtered scatter plot.

By plotting the scatter plot of the median house value and the median income, we can see that some of the median house values are capped. Likely an artifact of data collection. Hence, the capped data points were filtered out from the dataset using Pandas. To prevent our ML model from learning these false patterns, which would damage the models ability to generalise to new data.

4 Pipeline, Feature Engineering and Transformation

Our dataset contains a text attribute "Ocean Promixity". These are then one-hot encoded to convert them into a vector form that an ML algorithm is capable of interpreting. This is crucial in cases such as this when the attribute categories have no clear ordering. Missing data is handled using an imputer which fills in the most frequent value from the rest of the dataset.

The numerical pipeline is composed of an imputer that fills missing values with the median value of the feature. A log transformer which is applied to certain features with a heavy tail, as this helps our model to generalise better. Finally a StandardScaler is applied to standardise the data, to make our model less vulnerable to statistical outliers, than it would be if we simply normalised.

5 Evaluating Candidate Models

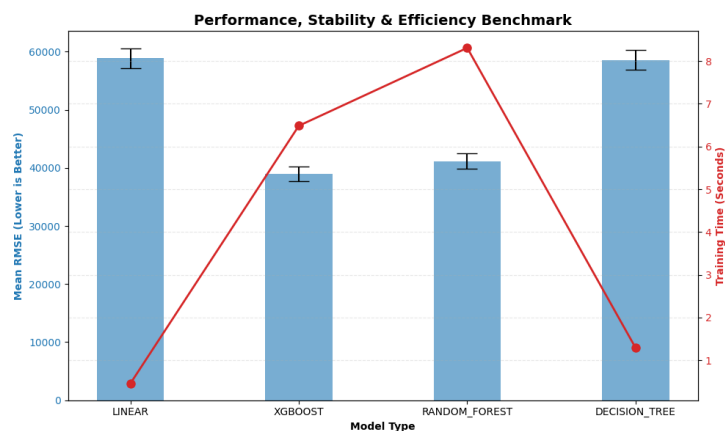


Figure 5: Fine-tuned the hyper-parameters of the XGBoost model using the RandomizedSearchCV SKLearn Class.

A selection of ML models (linear regression, XGBoost, Random Forest and Decision Tree) were trialed so see which performed best for this particular task. Their RMSE values and time taken to train were plotted to determine the optimal model. The models were each trained 5 times using cross validation to ensure the models generalised to new data aswell as possible.

6 Fine Tuning

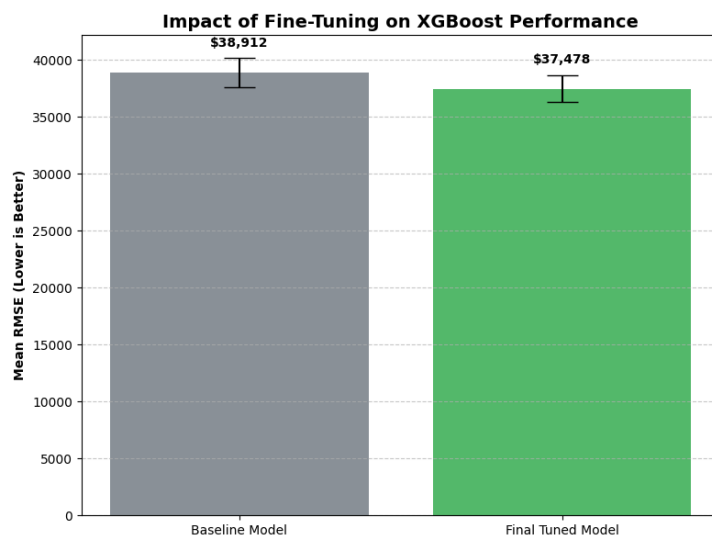


Figure 6: Fine-tuned the hyper-parameters of the XGBoost model using the RandomizedSearchCV SKLearn Class.

The XGBoost model was found to have the smallest RMSE. Thus it was chosen as the preferred model and prior to deploying it onto the test data. Its hyperparameters were finetuned using RandomizedSearchCV . This optimisation reduced the training RMSE value from \$38,912 to \$37,48. An improvement of \$1,434.

7 Testing and Results

This fine-tuned XGBoost model was applied to the 20% of the available data, which was withheld as the test dataset. The model had an **RMSE** = \$37,800.84 **with a 95% confidence interval between** [\$36,061.57, \$39,463.52]. Hence we can see that the model generalised to new data quite well, with the RMSE only increasing by \$320 dollars from the training result.